Recursive Algorithm Strategies

Recursion involves solving a problem by breaking it down into smaller and simpler instances of the same problem. Before you start designing a recursive algorithm for a problem, you must be clear on the following:

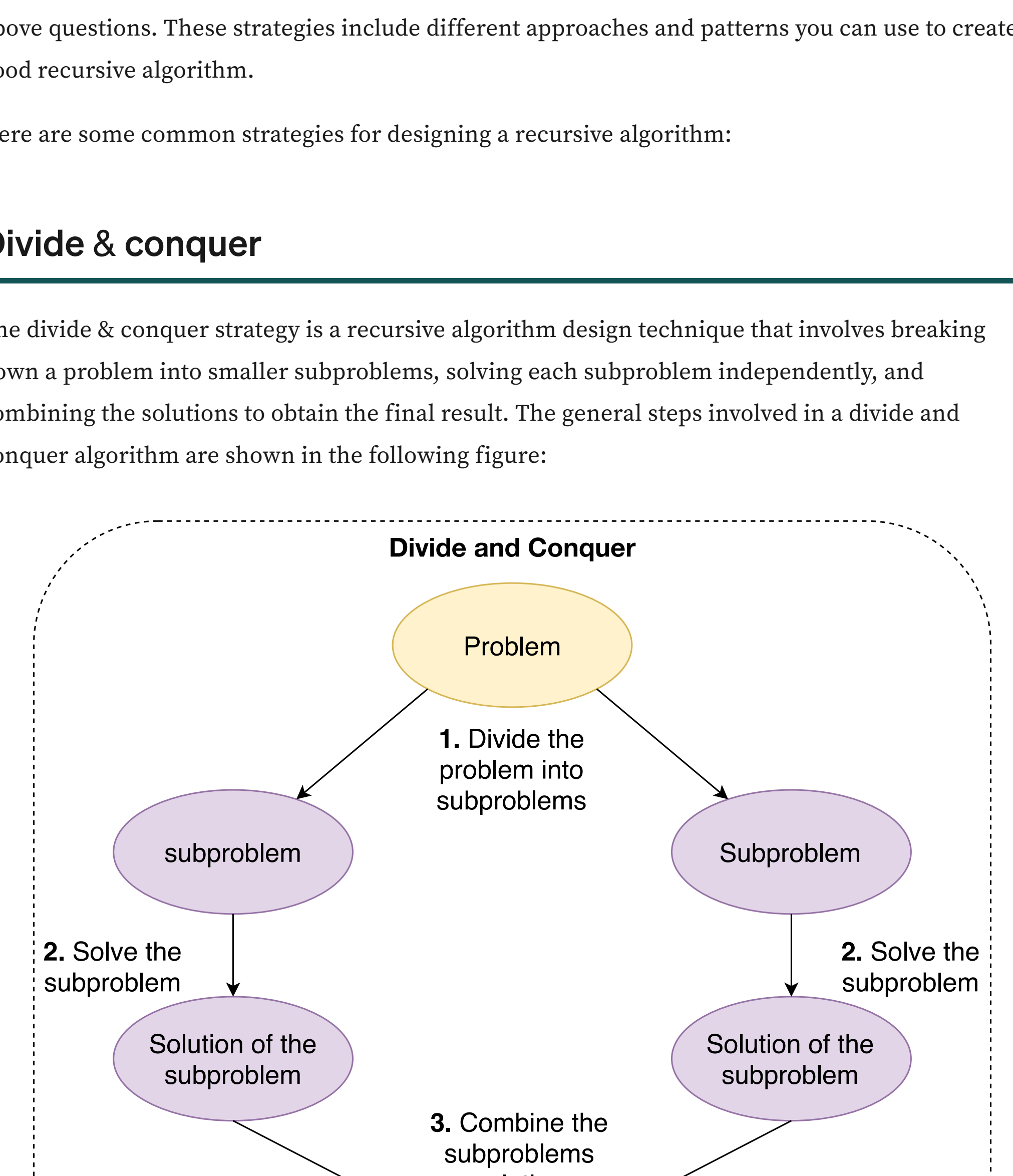
1. What and how to divide?
2. What and how to combine sub-problems to solve the bigger ones?
3. During computation, if the algorithm encounters a similar sub-problem again, will the algorithm solve it again?

The recursive algorithm strategies are classic frameworks that can help you find answers to the above questions. These strategies include different approaches and patterns you can use to create a good recursive algorithm.

Here are some common strategies for designing a recursive algorithm:

Divide & conquer

The divide & conquer strategy is a recursive algorithm design technique that involves breaking down a problem into smaller subproblems, solving each subproblem independently, and combining the solutions to obtain the final result. The general steps involved in a divide and conquer algorithm are shown in the following figure:


Divide and Conquer
Problem
1. Divide the problem into subproblems
subproblem
Subproblem
2. Solve the subproblem
2. Solve the subproblem
Solution of the subproblem
Solution of the subproblem
3. Combine the subproblems solution
Solution of the Problem

Here is a brief explanation of each step of this strategy:

1. **Divide**: Break the original problem into smaller, more manageable subproblems. This step can often be represented by recursively calling the same algorithm on the subproblems.
2. **Conquer**: Solve the subproblems independently. If the subproblems are small enough, they can be solved directly using a base case or a simple algorithm.
3. **Combine**: Combine the subproblems' solutions to obtain the final solution to the original problem. This step may involve merging or aggregating the subproblem solutions or applying additional operations to obtain the desired result.

Classic problems solved by divide & conquer strategy

The divide & conquer is suitable for problems that exhibit overlapping subproblems and can be efficiently solved by combining the solutions of smaller instances. Here are some classic problems that normally use the divide & conquer strategy in their recursive solution:

1. **Merge Sort**: A sorting algorithm divides the input array into two halves, recursively sorts each half, and then merges the two sorted halves to obtain a sorted array.
2. **Quick Sort**: Another sorting algorithm partitions the input array based on a selected pivot element, recursively sorts the subarrays on either side of the pivot, and combines them to get the final sorted array.
3. **Binary Search**: It is an efficient search algorithm for finding a target value in a sorted array. The algorithm repeatedly divides the search space in half and narrows down the search range based on the comparison of the target value with the middle element.
4. **Strassen's Matrix Multiplication**: This algorithm is an efficient way of multiplying matrices by recursively dividing them into smaller submatrices and combining the results using a set of mathematical operations.
5. **Closest Pair**: A problem involves finding the two closest points among a set of points in a two-dimensional space. The divide and conquer strategy can be applied by recursively dividing the points and merging the solutions to find the closest pair.

One should prefer divide and conquer strategy when the problem size significantly reduces with each division and combining sub-problems can solve larger problems. However, it's important to analyze the problem and ensure that the combination step does not introduce significant overhead, as it can affect the overall efficiency of the algorithm.

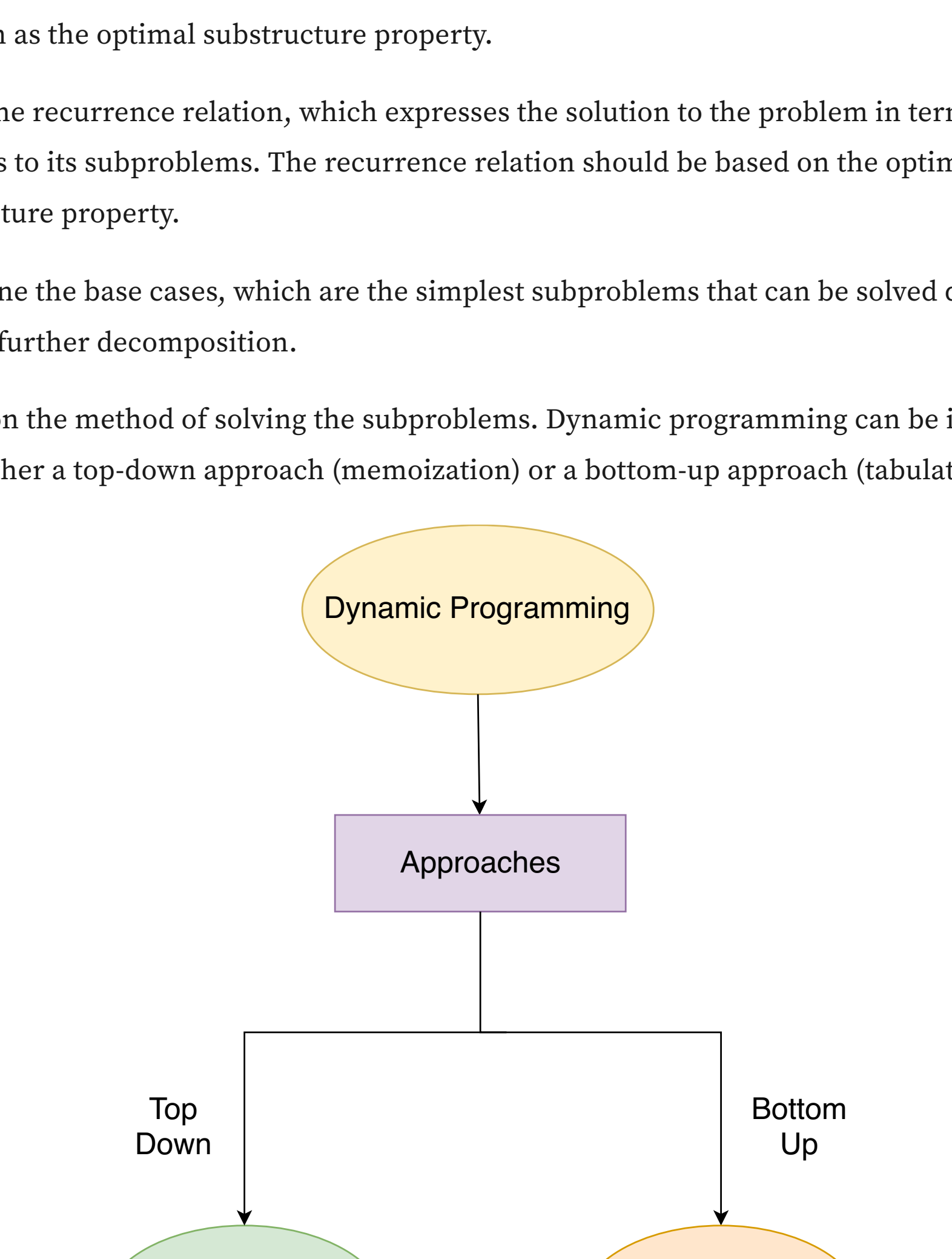
Dynamic programming

Remember this question that we posed in the start of this section - should we compute the sub-problems again and again on their subsequent encounters? While designing a recursive algorithm, if your answer to this question is "No" then you better use this dynamic programming strategy.

Dynamic programming is a technique used to solve complex problems by breaking them down into overlapping subproblems and **solving each subproblem only once.** It is often used to optimize recursive algorithms by storing and reusing the results of subproblems.

Dynamic programming typically follows these steps:

1. Identify the problem that can be divided into smaller, overlapping subproblems. This property is known as the optimal substructure property.
2. Define the recurrence relation, which expresses the solution to the problem in terms of solutions to its subproblems. The recurrence relation should be based on the optimal substructure property.
3. Determine the base cases, which are the simplest subproblems that can be solved directly without further decomposition.
4. Decide on the method of solving the subproblems. Dynamic programming can be implemented using either a top-down approach (memoization) or a bottom-up approach (tabulation).



- **Memoization**: In the top-down approach, the recursive algorithm is enhanced with a memoization techn ique. The results of subproblems are stored in a cache or a table to avoid redundant computations. Before solving a subproblem, the algorithm checks if its result is already available in the cache. If so, the cached result is used instead of re-computing it.
- **Tabulation**: In the bottom-up approach, the algorithm solves the subproblems iteratively, starting from the base cases and progressively building up to the final solution. The results of subproblems are stored in a table or an array, and each subproblem is solved only once.

5. Determine the order of solving the subproblems. The order can vary depending on the problem, but it should ensure that all the dependencies of a subproblem have been solved before solving that subproblem.
6. Compute and store the results of subproblems based on the recurrence relation, either using memoization or tabulation.
7. Finally, return the solution to the original problem, which is typically the result stored in the table or cache for the largest subproblem.

Dynamic programming is commonly used in various problem domains, such as optimization, graph algorithms, sequence alignment, and many others. By solving each subproblem only once and reusing the results, dynamic programming can significantly improve the efficiency of algorithms.

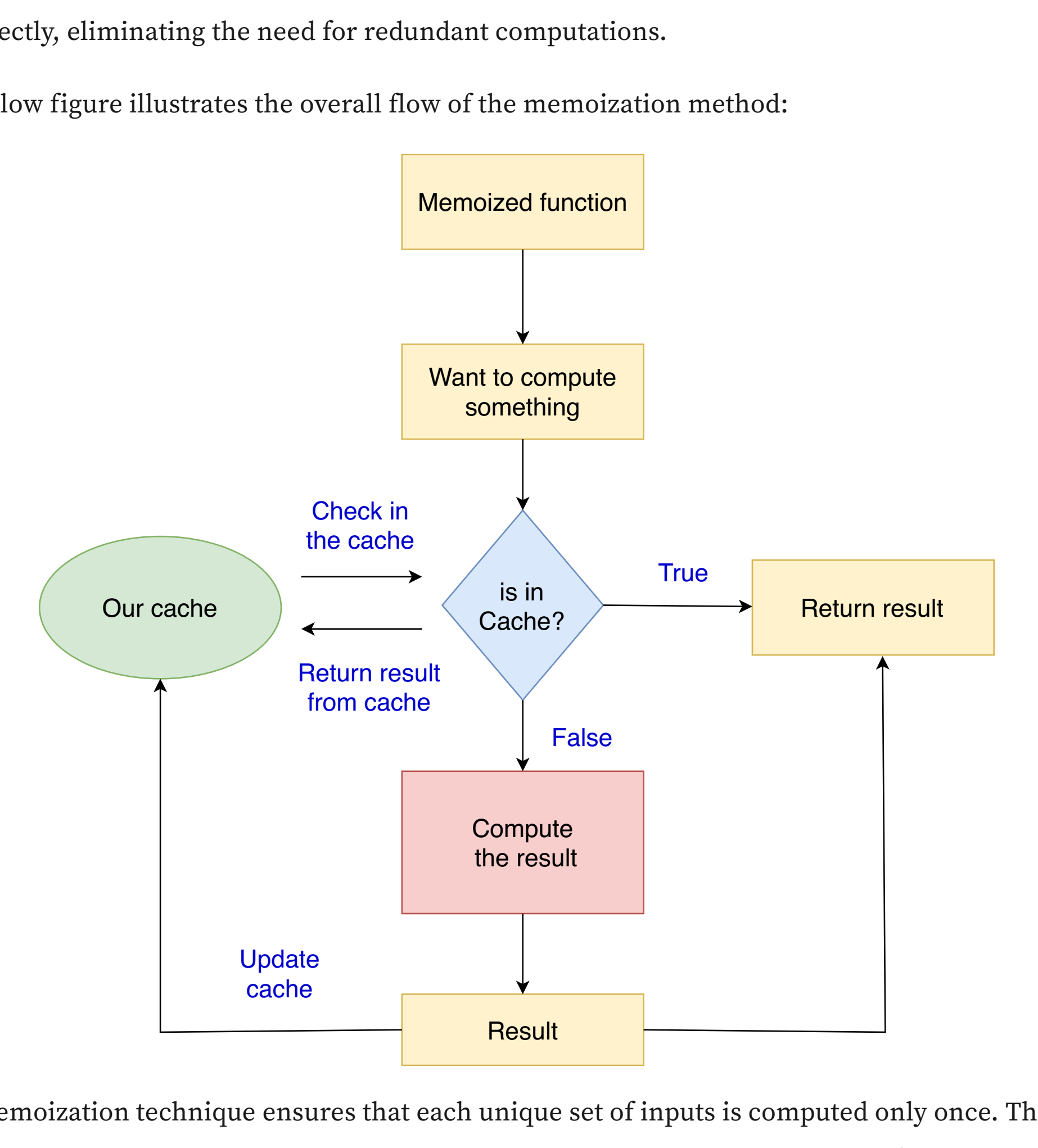
Let's discuss dynamic programming approaches in detail:

Memoization (top-down approach)

Memoization is a technique used to optimize the performance of recursive algorithms by caching the results of expensive function calls and reusing them when the same inputs occur again. It is often employed in dynamic programming to avoid redundant computations. The memoization technique can be summarized in the following steps:

1. Create a data structure, typically an array or a hash map, to store the computed results. The keys of the data structure are the inputs to the function, and the values are the corresponding outputs.
2. Before making a recursive function call, check if the result for the given inputs is already present in the cache. If it is, return the cached result instead of performing the computation again.
3. If the result is not in the cache, compute the result as usual and store it in the cache before returning it.
4. With each subsequent function call, check the cache first. If the result is found, return it directly, eliminating the need for redundant computations.

The below figure illustrates the overall flow of the memoization method:



The memoization technique ensures that each unique set of inputs is computed only once. This becomes possible because the subsequent calls with the same inputs can directly fetch the cached result. Thereby, it leads to significant performance improvements in scenarios where the same function is called with the same inputs repeatedly.

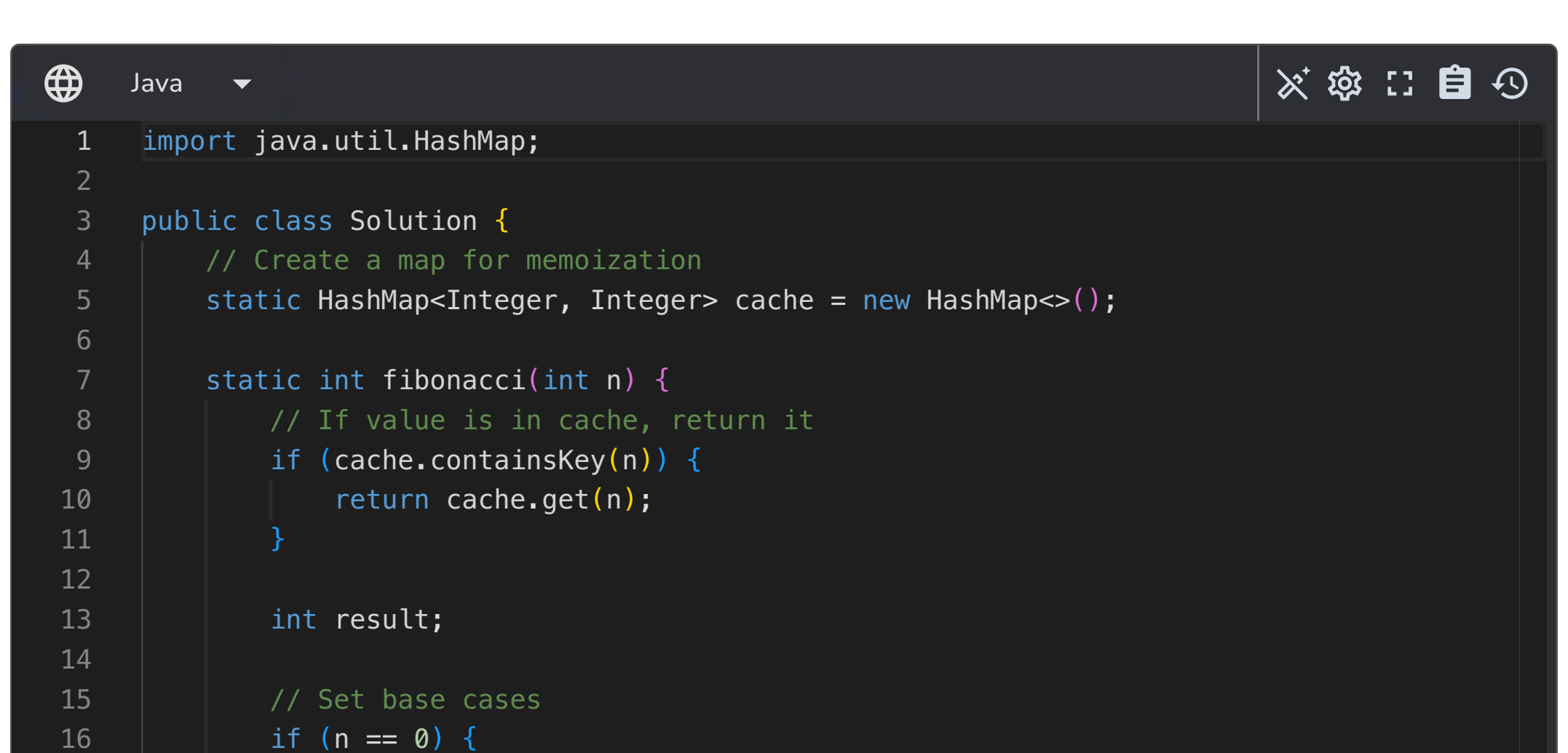
Here's an example to demonstrate the application of memoization:

```
function fibonacci(n, cache):
    if n is in cache:
        return cache[n]

    if n equals 0:
        result = 0
    else if n equals 1:
        result = 1
    else:
        result = fibonacci(n-1) + fibonacci(n-2)
    cache[n] = result
    return result
```

In the above example, the `fibonacci` function uses memoization to compute Fibonacci numbers. The cache, represented by the `cache` dictionary, stores the computed Fibonacci numbers as they are calculated. Whenever the function is called with a specific input `n`, it first checks if the result is already cached. If so, it directly returns the cached result. Otherwise, it computes the result recursively, stores it in the cache, and then returns it.

Here is the complete code for computing `nth` Fibonacci number using memoization:



```java
import java.util.HashMap;

public class Solution {
    // Create a map for memoization
    static HashMap<Integer, Integer> cache = new HashMap<>();

    static int fibonacci(int n) {
        // If value is in cache, return it
        if (cache.containsKey(n)) {
            return cache.get(n);
        }

        int result;

        // Set base cases
        if (n == 0) {
            result = 0;
        }
        else if (n == 1) {
            result = 1;
        }
```

Run Code Result Stdout

You must run your code first

Execute

By memoizing the intermediate results, the function avoids redundant computations and significantly improves the performance for larger values of `n`. Without memoization, the recursive Fibonacci algorithm would have an exponential time complexity, but with memoization, it effectively reduces it to linear time complexity.

Note that when using memoization, it's important to be mindful of the cache management and its impact on memory usage. In some cases, it may be necessary to limit the size of the cache or clear it when needed to avoid excessive memory consumption.

Tabulation (bottom-up approach)

The bottom-up approach, also known as tabulation, is a technique used in dynamic programming to solve problems iteratively by building solutions from the bottom up. Instead of using recursion and memoization as in the top-down approach, the bottom-up approach avoids recursive function calls and computes the solutions iteratively in a systematic manner.

Here's how the bottom-up approach works:

1. **Identify the subproblems**: Analyze the problem and break it down into smaller overlapping subproblems. Each subproblem should have a well-defined input and output.
2. **Define a table**: Create a table (usually an array or a matrix) to store the solutions to the subproblems. The table size is determined based on the input size and the number of subproblems.
3. **Initialize base cases**: Fill in the table with the solutions to the base cases, which are the smallest subproblems that can be directly solved.
4. **Build solutions iteratively**: Use the values from previously solved subproblems to solve larger subproblems. Iterate through the table in a bottom-up manner, solving each subproblem and storing its solution in the table.
5. **Return the final solution**: Once all subproblems are solved, the solution to the original problem will be stored in the table at the appropriate position.

The bottom-up approach is often preferred when implementing dynamic programming algorithms because it eliminates the overhead of function calls and avoids issues with recursion depth. It also guarantees that all necessary subproblems are solved, as the solutions are computed in a systematic and incremental manner.

Let's illustrate the bottom-up approach with an example using the Fibonacci sequence:

```
function fibonacci(n):
    fibTable = array of size n+1

    # Initialize base cases
    fibTable[0] = 0
    fibTable[1] = 1

    # Build solutions iteratively
    for i = 2 to n:
        fibTable[i] = fibTable[i-1] + fibTable[i-2]

    return fibTable[n]
```

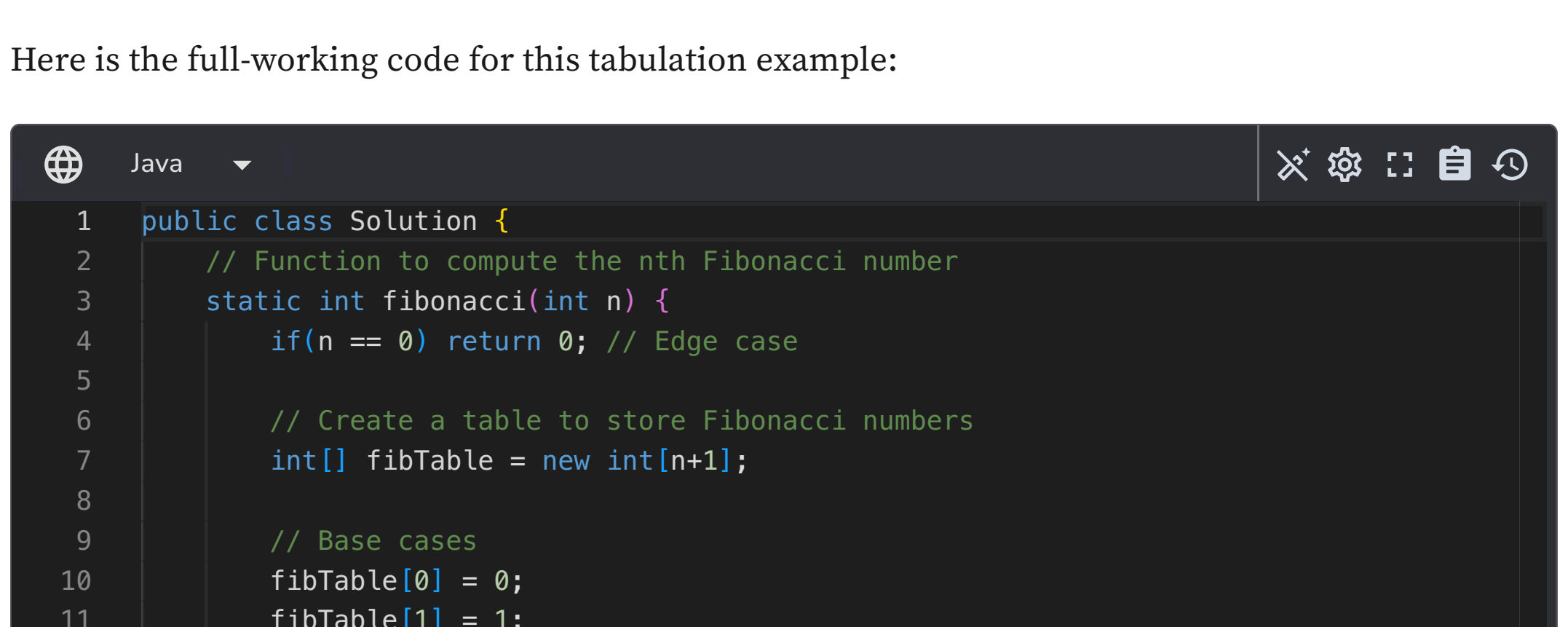
In the above pseudo-code, the function `fibonacci(n)` takes an input `n` and computes the nth Fibonacci number using the bottom-up approach. It initializes an array `fibTable` of size n+1 to store the Fibonacci numbers.

The base cases `fibTable[0]` and `fibTable[1]` are set to 0 and 1, respectively, as they are known values in the Fibonacci sequence.

The for loop iterates from 2 to n, computing each Fibonacci number by summing the two previous Fibonacci numbers (`fibTable[i−1]` and `fibTable[i−2]`) and storing it in `fibTable[i]`.

Finally, the function returns the nth Fibonacci number, which is accessed through `fibTable[n]`.

Here is the full-working code for this tabulation example:



```java
public class Solution {
    // Function to compute the nth Fibonacci number
    static int fibonacci(int n) {
        if(n == 0) return 0; // Edge case

        // Create a table to store Fibonacci numbers
        int[] fibTable = new int[n+1];

        // Base cases
        fibTable[0] = 0;
        fibTable[1] = 1;

        // Fill fibTable in bottom−up manner
        for(int i = 2; i <= n; i++) {
            fibTable[i] = fibTable[i−1] + fibTable[i−2];
        }

        // Return nth Fibonacci number
        return fibTable[n];
    }

```

Run Code Result Stdout

You must run your code first

Execute

By using the bottom-up approach, dynamic programming problems can be efficiently solved without recursion and with a clear and straightforward iterative process.

The below table shows the comparison between tabulation and memoization approaches of dynamic programming:

Technique	Tabulation	Memoization
Overview	Iteratively builds solutions bottom-up	Stores computed solutions for reuse top-down
Dependency	Solves subproblems before solving larger problems	Retrieves solutions from cache when needed
Data Structure	Typically uses arrays or matrices for tabular storage	Utilizes data structures like arrays, hash maps, or caches
Order of Access	Iterative order of accessing subproblems	Recursive order of accessing subproblems
Space Complexity	Generally requires more space for storing the table	Requires space for cache storage
Time Complexity	Generally efficient with optimized time complexity	May have overhead due to function calls and cache lookups
Initialization	Requires initializing the table with base case values	Requires setting up the cache with initial values
Usage	Suitable for problems where subproblem dependencies are well-defined and can be computed independently	Suitable for problems with overlapping subproblems or repetitive computations
Implementation	Often involves nested loops for iterating over the table	Requires modifying recursive functions to include caching
Examples	Dynamic programming problems like Fibonacci, knapsack	Memoizing recursive functions for Fibonacci, factorial

← Previous
Recursion Vs. Iteration
Next →
Complexity Analysis
☐ Mark as Completed